Addition 7: Live On-the-Go Self-Calibrating Loop

Addition 1 (`online_predictor.py`) validated the *logic* of this loop in simulation and found one component genuinely works: the hardware-distance confidence check correctly flags a hardware outlier (c5a) and correctly resolves to "known" the moment that hardware region has been observed at all. Addition 1's *combined* three-component gate (which also weighted a quantile-interval-width signal) did not improve overall accuracy — that interval component was diagnosed as unstable and not something to build on. This addition wires the validated half — hardware-distance confidence — into the actual gate (`cetp_gate.py`'s existing `hardware_distance_confidence`, unchanged) as a persistent, live mechanism, not a new simulation.

The datastore

`measurement_store.py`, SQLite, file-based, no server. Default path `cetp_measurements.db` at the project root (the demo below uses its own `cetp_measurements_addition7_demo.db` so it can be deleted and rebuilt from scratch on every run without touching a real deployment's store).

Schema (three tables):

```
machines (machine_id PK, bandwidth, compute, source['seed'|'captured'], added_at)
measurements (id PK, machine_id FK, query_id, cost, shared_hit, shared_read, rows, time_ms,
source['seed'|'captured'], captured_at)
retrain_log (id PK, retrained_at, model_version, n_captured_total,
n_captured_since_prior_retrain, machines_snapshot)
```

`init_store()` seeds `machines` from the static hardware-signature table and `measurements` from `tpch_dataset_corrected.csv` (the buffer-bug-corrected dataset from the prior addition — used deliberately as the more accurate seed) aggregated to one row per (machine, query). This is the single source of truth every retrain reads from; nothing else feeds the model.

Entry points

- `record_measurement(machine_id, query_id, cost, shared_hit, shared_read, rows, time_ms, bandwidth=None, compute=None, db_path=...)` — appends one real measurement. If `machine_id` is new, it must be registered with its own static `(bandwidth, compute)` hardware signature (a separate, one-time hardware benchmark) — a captured `prod_time_ms` can never itself supply that signature, which is the anti-leakage boundary made structural, not just a rule. In a real deployment this is called after an actual prod run; the demo below calls it with a held-out row from the corrected dataset standing in for that measurement, and says so.
- `retrain(db_path=..., n_trigger=20, force=False, persist=True)` — counts captured measurements since the last retrain; if `>= n_trigger` (or `force=True`), rebuilds the aggregated feature table from *all* measurements (seed + captured), refits the bottleneck classifier and the three quantile GBR models (`random_state=42`), rebuilds the hardware-signature table from the `machines` table (so any newly-registered machine is now in it), writes a `retrain_log` row, persists versioned artifacts to `models_online/v<N>/`, prints a log line, and returns the new model set. Never a silent side effect — it's explicit, logged, and returns `None` if the threshold isn't met.

`cetp_gate.py` was refactored (no behavior change, all 28 existing tests still pass) to expose `run_gate_with_models(...)`, which takes model objects directly instead of loading `models/*.pkl` from disk — this is what lets the demo feed freshly retrained in-memory models through the exact same gate logic (`bottleneck_probabilities`, `predict_scaling`, `hardware_distance_confidence`, `three_state_verdict`) without touching the shipped production artifacts.

The loop-closing demonstration (`addition7_online_loop.py`)

Setup: 4-machine world (z1d, r5n, m5a, c7i), c5a — the established Phase 7 hardware outlier — entirely unseen. 9 of c5a's 21 queries, spanning all four bottleneck classes, are reserved and never fed to `record_measurement` (`RESERVED_TEST_QUERIES`), so the "after" evaluation is on genuinely held-out queries, not the ones just trained on. The remaining 12 queries are revealed one at a time via `record_measurement` (`RETRAIN_TRIGGER_N = 12`).

Request 1 (dev=c7i, prod=c5a, q19, before any c5a data):

```
state=WARN  confidence=0.000
reason=LOW CONFIDENCE (hw-distance confidence=0.00 < 0.6): prod hardware signature is an
outlier
       relative to the training set. Recommend a canary run or real measurement instead of
trusting
       this prediction.
```

Assertion `state == "WARN" and low_confidence` passes — the system correctly declines to trust itself.

Measure and capture: 12 real c5a measurements revealed and recorded (first call also registers c5a's static hardware signature, bandwidth=14.9, compute=0.593). `retrain(n_trigger=12)` fires: `models_online/v2/` persisted, `known machines = ['c5a', 'c7i', 'm5a', 'r5n', 'z1d']`.

Request 2 (same dev/prod/query, after learning):

```
state=PASS  confidence=1.000
reason=predicted p99 (82.8 ms) clears SLA.
```

Assertions pass: confidence rose (0.000 → 1.000) and `low_confidence` is now `False`.

Before / after, the numbers

	confidence (c5a)	narrative-query error (c7i→c5a, q19)
before learning	**0.000**	predicted 1.2575 vs actual 0.8227 → **52.8%** error
after learning	**1.000**	predicted 0.6745 vs actual 0.8227 → **18.0%** error

Held-out accuracy across all 9 reserved queries × 4 dev machines (n=36 dev→c5a pairs, none of which were used in retraining):

	mean confidence	MAPE
before learning	0.000	**77.72%**
after learning	1.000	**15.01%**
improvement		**+62.71 pp**

Confidence goes to exactly 0.000 before (c5a's bandwidth, 14.9, sits outside the box spanned by the 4 known machines' bandwidths [8.44, 10.01], pushing normalized distance past `sqrt(2)`, clipped at 0) and exactly 1.000 after (distance to itself, once c5a is in the table, is 0). Both are the correct, mechanically-expected values, not artifacts.

Confirmed: after retraining, the machine that was flagged LOW-confidence is now treated as in-distribution and returns a real (PASS/BLOCK/WARN-on-SLA, not WARN-on-confidence) verdict — not a defer.

Leakage re-check

Ran against the actual retrained pipeline (`addition7_online_loop.py`'s `leakage_check()`): `SCALING_FEATURE_COLUMNS` and `RAW_BOTTLENECK_COLS` — the only two feature sets the retrained models ever see — contain no `prod_*` column. Source-inspected `measurement_store._build_pairs`: `prod_time_p50` is read exactly twice per pair — once to become a *different* row's `dev_time_p50` when that machine plays dev elsewhere, and once divided by `dev_time_p50` to build `scaling_factor_p50`, the regression **label**. It never enters the `X` matrix passed to `GradientBoostingRegressor.fit()`. **Result: PASS.** Captured prod times enter the retrained model only as labels, exactly as in the original validated pipeline.

Honest characterization

This demonstrates the mechanism — detect unfamiliar hardware, defer, capture a measurement, retrain, and have that machine become known with materially better accuracy — using held-out rows from the corrected dataset as a stand-in for a fresh measurement. **It has not been tested against a live production stream.** A real deployment triggers `record_measurement` from an actual prod run, not from revealing existing rows. The before/after evidence here (0.000 → 1.000 confidence, 77.72% → 15.01% held-out MAPE) is real and reproducible (all randomness seeded; rerunning `addition7_online_loop.py` from a deleted store reproduces the same numbers to the decimal), but it is evidence that the wiring and the mechanism work on this dataset, not evidence of production validation.

Artifacts

- `measurement_store.py` — schema, `init_store`, `record_measurement`, `retrain`.
- `cetp_gate.py` — added `run_gate_with_models` (existing `run_gate` unchanged in behavior; all 28 tests in `test_e2e.py` still pass).
- `addition7_online_loop.py` — the demonstration harness.

- `cetp_measurements_addition7_demo.db` — the demo's persistent store (rebuilt from `tpch_dataset_corrected.csv` on each run).
- `models_online/v2/` — the retrained, persisted model snapshot.
- `results/addition7_before_learning_eval.csv`, `results/addition7_after_learning_eval.csv`, `results/addition7_summary.json`.